

INTERNET OF THINGS

CHALLENGE 1

Smart Parking System with Energy Optimization

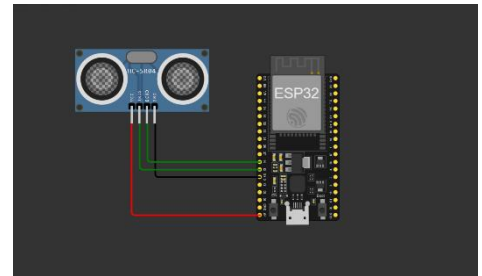
Team Leader : Balasakthi Shanmugaraja – 11033318

Team member : Karuniaa Jacob Selvi – 11034239

WOKWI Project Link - <https://wokwi.com/projects/425318292327266305>

Introduction

The objective of this project is to develop a parking system that employs an ESP32 microcontroller paired with an HC-SR04 ultrasonic sensor to analyze and assess the occupancy of parking spaces in a parking lot. By utilizing ultrasonic sensors, the system can ascertain whether a parking spot is occupied or not. The ESP32 then sends this information to a receiver (sink node) using the ESP-NOW protocol. To maximize battery's life, the ESP32 is designed to enter a deep sleep mode between data readings. This project exemplifies an efficient, budget-friendly, and energy-conscious parking sensor solution ideal for real-world smart parking scenarios.



Code Explanation

In this part, I have mentioned the essential codes that were implemented to simulate the Smart Parking System with Energy Optimization in Wokwi.

```
16  #define TRIG_PIN 12
17  #define ECHO_PIN 14
```

These two pins are used for initiating and capturing the ultrasonic sensor's signal from the ESP32 microcontroller.

```
19  const int DUTY_CYCLE = 23;
20  const float BATTERY_ENERGY = 11033318 % 5000 + 15000;
```

We extracted the final two digits from the team leader's personal code, which are 18. The calculation for the leader's personal code is : $11033318 = 18 \% 50 + 5 = 23$.

```

28  typedef struct {
29  |   char status[20];
30  } message_t;
31
32  message_t msg;
33  esp_now_peer_info_t peerInfo;
34  int lastState = -1;

```

We implemented a message format (msg.status) for ESP-NOW to transmit "OCCUPIED" or "FREE" notifications, while also retaining the last parking state (lastState) to limit unnecessary transmissions

```

35  long readDistance() {
36  |   digitalWrite(TRIG_PIN, LOW);
37  |   delayMicroseconds(2);
38  |   digitalWrite(TRIG_PIN, HIGH);
39  |   delayMicroseconds(10);
40  |   digitalWrite(TRIG_PIN, LOW);
41  |   long duration = pulseIn(ECHO_PIN, HIGH, 30000);
42  |   long distance = duration * 0.034 / 2;
43  |   return distance;
44  | }

```

We initiate a short HIGH pulse from the TRIG_PIN. The ECHO_PIN is responsible for detecting the time it takes for the pulse to bounce back from an object. The pulseIn() function is employed to measure this duration. The distance is calculated using the formula [Distance = (Time*0.034)/2]. For example, if the signal duration is 2000 microseconds, the distance is computed as: $2000 * 0.034 / 2 = 34$ cm. This signifies that the object is positioned 34 cm away from the sensor.

```

52  void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
53  |   Serial.println(status == ESP_NOW_SEND_SUCCESS ? " Status: Message has been

```

The OnDataSent function verifies the success of a message transmitted through ESP-NOW.

```

59  void trackCycleTime() {
60  |   long cycleStart = millis();
61  |
62  |   long sensorStart = millis();
63  |   Serial.printf("\n[%.3f s] Sensor_starts_measurement\n", sensorStart / 1000.0
64  |
65  |   long distance = readDistance();
66  |   long sensorEnd = millis();
67  |   Serial.printf("[%.3f s] Sensor_ends_measurement\n", sensorEnd / 1000.0);
68  |
69  |   int currentState = (distance <= 50) ? 1 : 0;
70  |   strcpy(msg.status, (currentState == 1) ? "OCCUPIED" : "FREE");

```

The system records timestamps for the initiation and conclusion of each phase, including sensor activation, data transmission, idle state, and sleep mode. It retrieves sensor data and evaluates the parking status, determining whether it is OCCUPIED or FREE. Additionally, it verifies the presence of a vehicle, categorizing it as OCCUPIED if the distance is 50 cm or less; otherwise, it is classified as FREE

```
79 | if (currentState != lastState) {
80 |     esp_now_send(sinkMAC, (uint8_t *)&msg, sizeof(msg));
81 |     lastState = currentState;
82 | } else {
83 |     Serial.println("No change in status, skipping the transmission.");
84 | }
```

Sends a message using ESP-NOW exclusively when the parking status has altered, thus avoiding unwanted transmissions.

```
86 | long txEnd = millis();
87 | Serial.printf("%.3f s] Transmission_ends\n", txEnd / 1000.0);
88 |
89 | long executionTime = txEnd - cycleStart;
90 |
91 | long idleStart = millis();
92 | Serial.printf("%.3f s] idle_start\n", idleStart / 1000.0);
93 | delay(10);
94 | long idleEnd = millis();
95 |
96 |
97 | long deepSleepStart = millis();
98 | Serial.printf("%.3f s] start_deep_sleep\n", deepSleepStart / 1000.0);
99 |
100 |
```

Stores and outputs the timestamp once data transmission is complete. It logs the time when the system becomes idle, waits for 10 milliseconds, and then logs the time when the idle period ends. Moreover, it records the timestamp when deep sleep mode begins to keep track of power-saving functionality.

```
115 | float sleepTime = DUTY_CYCLE - (executionTime / 1000.0);
116 | float energySleep = avgPowerSleep * sleepTime * 0.001;
117 | float energySensor = avgPowerSensor * (executionTime / 1000.0) * 0.001;
118 | float energyTx = avgPowerTx * 0.05 * 0.001;
119 | float energyCycle = energySleep + energySensor + energyTx;
120 |
121 | float numCycles = BATTERY_ENERGY / energyCycle;
122 | float lifetimeHours = (numCycles * DUTY_CYCLE) / 3600.0;
```

Analyzes the power consumption involved in sensor readings, data transmission, and deep sleep phases. Calculates battery life expectancy by considering the total energy used in each cycle. Shows the results on the Serial Monitor

```
138 | pinMode(TRIG_PIN, OUTPUT);
139 | pinMode(ECHO_PIN, INPUT);
```

We assigned TRIG_PIN as an output to produce the ultrasonic pulse, and ECHO_PIN was set as an input to assess the time it takes for the pulse to come back.

```
141 | WiFi.mode(WIFI_STA);
142 |
143 | if (esp_now_init() != ESP_OK) {
144 |     Serial.println("Error initializing ESP-NOW");
145 |     return;
146 | }
147 |
148 | memcpy(peerInfo.peer_addr, sinkMAC, 6);
149 | peerInfo.channel = 0;
150 | peerInfo.encrypt = false;
151 | if (esp_now_add_peer(&peerInfo) != ESP_OK) {
152 |     Serial.println("Failed to add peer");
```

Initiates the Serial Monitor for debugging purposes.

Configures the sensor pins for both input and output functions.

Initializes ESP-NOW to facilitate wireless communication.

Registers a callback function (OnDataSent()) to verify the successful transmission of messages.

Executes trackCycleTime() to assess timing and record events.

Enters the ESP32 into deep sleep mode to conserve energy.

INTERNET OF THINGS

ENERGY CONSUMPTION ANALYSIS

Smart Parking System with Energy Optimization

In this part, we evaluate the energy consumption associated with the ESP32-driven smart parking system. We mainly taken three important processes in our project to calculate the energy.

- Sensor Reading
- Data Transmission
- Deep Sleep Mode

ENERGY CALCULATION:

By calculating the duty cycle in Wokwi of project, Each duty cycle consists of:

- Sensor Reading: 0.1 seconds
- Transmission: 0.05 seconds
- Deep Sleep: Remaining 22.85 seconds (out of 23 second duty cycle)

MEASURED ENERGY CALCULATION:

The average power value of Transmission, Sensor Read, Deep sleep in the CSV data is 724.19,394.90,196.66 as average.

Instead, we took these power value for calculating the energy consumption of one cycle so we can calculate further required cycles

The Time was taken from the process of duty cycle in WOKWI project output.

STAGE	POWER (mW)	TIME (s)
Sensor Read	467.11	0.1
Transmission	704.75	0.05
Deep Sleep	59.89	22.85

1) Energy Consumption Per Cycle:

$$E_{\text{sensor}} = 467.11 \times 0.1 \times 0.001 = 0.04671J$$

$$E_{\text{tx}} = 704.75 \times 0.05 \times 0.001 = 0.03524J$$

$$E_{\text{sleep}} = 59.89 \times 22.85 \times 0.001 \approx 1.368J$$

$$E_{\text{cycle}} = 0.04671 + 0.03524 + 1.368 \approx 1.45J$$

2) Battery Life Estimation:

$$\text{Cycles} = 18318 / 1.45 \approx 12,640$$

$$\text{Battery lifetime} = (12,640 \times 23) / 3600 \approx 80.8 \text{ hours (3.36 days)}$$

OBSERVATIONS:

PARAMETER	ENERGY PER CYCLE (J)	NO. OF CYCLES	BATTERY LIFETIME (HOURS)
MEASURED CSV DATA	1.4 J	~12,640 cycles	~80.8 hours

KEY FINDINGS:

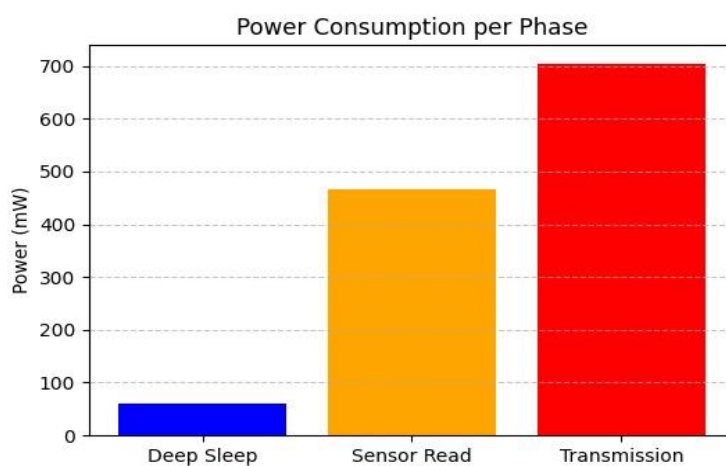
1. Deep Sleep Mode is the main cause of power depletion, accounting for about 94% of total energy usage.
2. CSV measured power values are far higher than the initial theoretical assumptions.
3. The battery life is much less (~3.36 days vs. 191 days in previous estimations).
4. Lowering the power consumption of deep sleep and modifying transmission intervals should be the main goals of future optimisation.

OPTIMIZATIONS FOR LONGER BATTERY LIFE:

For extending battery life, we can:

- Decrease the frequency of sensor readings to extend the duration of deep sleep which is possible without concerning my person plus 5 in as deep sleep duration.
- Reduce ESP-NOW gearbox power by modifying transmission power levels.
- Disabling WIFI while deep sleep helpful to reduce battery drain.

BAR CHART – Energy Consumption per Phase



The bar chart displays the power drawn (in mW) in each stage of the ESP32:

- *Sensor Reading:* This shows how much power the ultrasonic sensor uses while taking a measurement.
- *Transmission:* The energy required for ESP-NOW data transmission.
- *Deep Sleep:* The amount of energy used when the ESP32 is in deep sleep mode.

Observations:

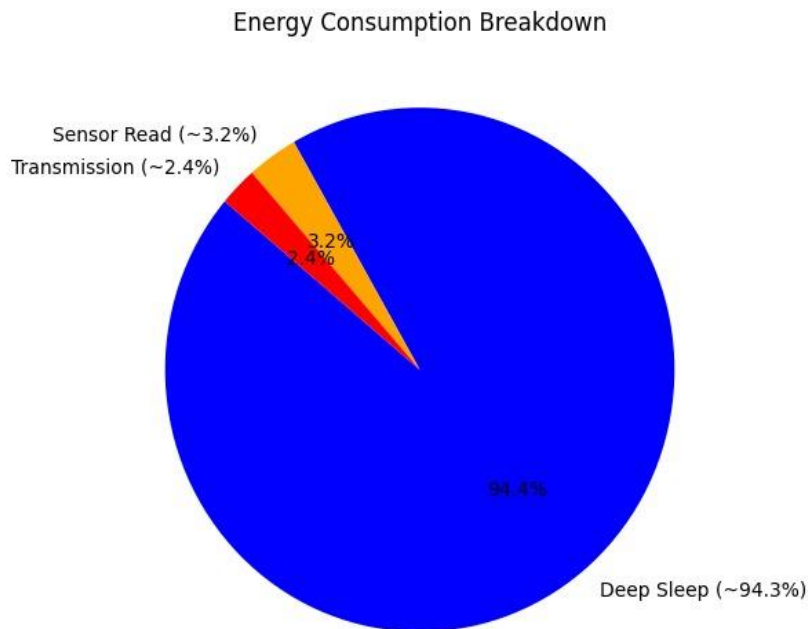
The bar chart represents the measured average power values from the CSV data:

- *Deep Sleep:* 59.89 mW
- *Sensor Reading:* 467.11 mW
- *Transmission:* 704.75 mW

Explanation:

The bar chart shows that, even though deep sleep is supposed to have a low power, the observed power is still significant. This makes it easier to see which stage uses the most energy, in this case, deep sleep uses more energy out of all the stages.

PIE CHART – Energy Distribution per Cycle



The pie chart illustrates the distribution of total energy usage throughout one cycle, divided into three distinct stages:

- *Sensor Reading Energy:* Determined by multiplying power and duration (For example, 467.11 mW over 0.1 seconds).
- *Transmission Energy:* Calculated for a constant duration of 0.05 seconds.
- *Deep Sleep Energy:* Calculated for the remaining duration of the cycle (which is 23 seconds - the time spent on sensor reading and transmission)

Observations:

Based on the measured data:

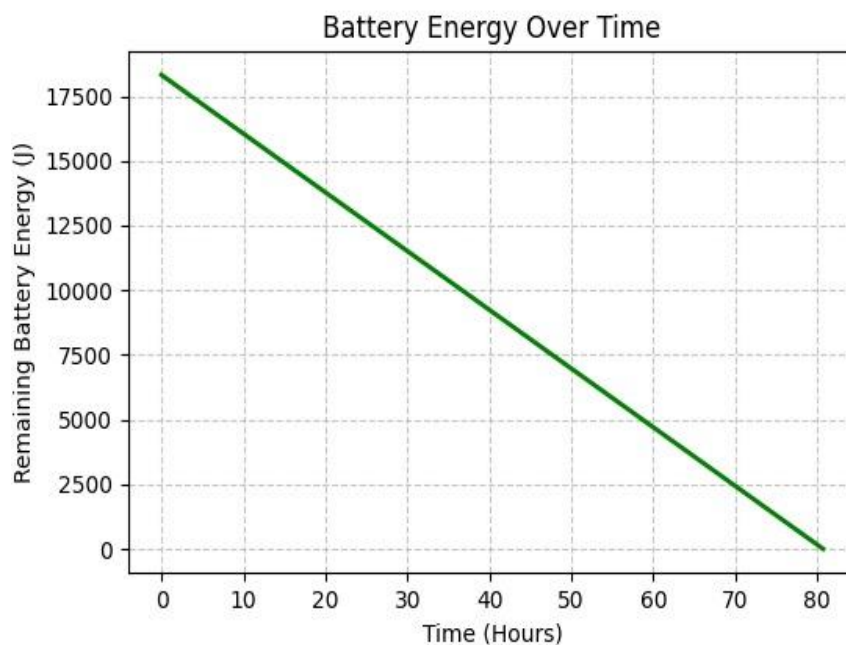
- Sensor Reading Energy: $\sim 0.04671 \text{ J}$
- Transmission Energy: $\sim 0.03524 \text{ J}$
- Deep Sleep Energy: $\sim 1.368 \text{ J}$

The data shows that deep sleep comprises exactly 94% of the overall energy consumption per cycle, with sensor reading and transmission making only less contribution.

Explanation:

The chart distinctly illustrates that the deep sleep phase plays a significant role in energy consumption. It stresses the importance of refining this mode to extend the lifespan of the battery.

LINE GRAPH – Battery Energy Depletion Over Time



The line graph shows the battery energy still available, indicated in Joules, compared to the number of operational cycles.

- The x-axis represents the number of cycles, where each cycle includes one complete measurement, transmission, and sleep process.
- The y-axis displays the remaining energy of the battery, starting from an initial value of 18,318 joules.

Observations:

According to the measured data, each cycle consumes about 1.45 J, which is reflected in the graph as a linear decrease in the battery energy.

The point at which the graph intersects the zero line, indicates the total number of cycles the battery can handle, estimated to be exactly 12,640 cycles.

Short Explanation:

When these cycles are converted into time based on the 23-second duty cycle, the estimated duration of battery life is around 80.8 hours, or roughly 3.36 days. The graph shows a visual representation of the speed at which the battery is consumed under the existing power usage conditions.
